Level 2

Hackathon – Fundamentals of Web Development

Personal Expenses Tracker

Student Details

Student Name	Aslam.A
Register Number	25131031802521009
Department	214 - B.Sc. COMPUTER SCIENCE
ID	35FD9BF3AC3EEA196FD4D8DE7F1F701C
College	103-GOVERNMENT ARTS COLLEGE-C.MUTLUR - CUDDALORE
Date of Submission	27/10/2026

Git-Hub Link:

https://github.com/aslam-oops/PERSONAL-EXPENSE-TRACKER-

1. Problem Statement

Many people forget where their money goes. They write expenses on paper, remember them, or see them in bank messages. Because of this, it is hard to know the balance, to see which category costs the most, or to compare one month with another. This leads to overspending.

The Personal Expense Tracker solves this problem. It is a simple web page where the user can save income and expenses with amount, date, category and description. The user can also search the records and see the total, a monthly summary and a donut chart. The data is saved in the browser (Local Storage), so no server or login is needed.

2. Objectives

- To add, edit and delete income and expense records.
- To group records with categories that the user can manage.
- To search and filter records by description, category, type and date.
- To show total income, total expenses and balance automatically.
- To show a monthly summary and a donut chart of expenses.
- To save data in Local Storage and update everything when data changes.

Expected result: an easy and fast web page that helps users understand their spending and plan their money better. It is made with only HTML, CSS and JavaScript.

3. Architecture

3.1 Application Structure

The application is a single web page that runs in the browser. It has three files:

- **index.html –** makes the page structure: cards, form, chart, tables and filters.
- **style.css –** gives the colours, layout and small-screen support.
- **script.js –** has all the logic: save data, check input, filter, calculate and draw the chart.

3.2 User Navigation Flow

The user opens the page and sees the dashboard. The user can add a record, edit or delete a record, add or delete a category, search and filter, and change the chart view. Every change is saved and the totals, summary, chart and table update at once.

3.3 Technologies Used

Technology	Use in the project
HTML5	Makes the page structure, form and tables.
CSS3	Gives colours, layout and small-screen design.
JavaScript	Does all the work: add, edit, delete, filter and calculate.
SVG	Draws the donut chart.
Local Storage	Saves data in the browser.

3.4 Application Flow Diagram

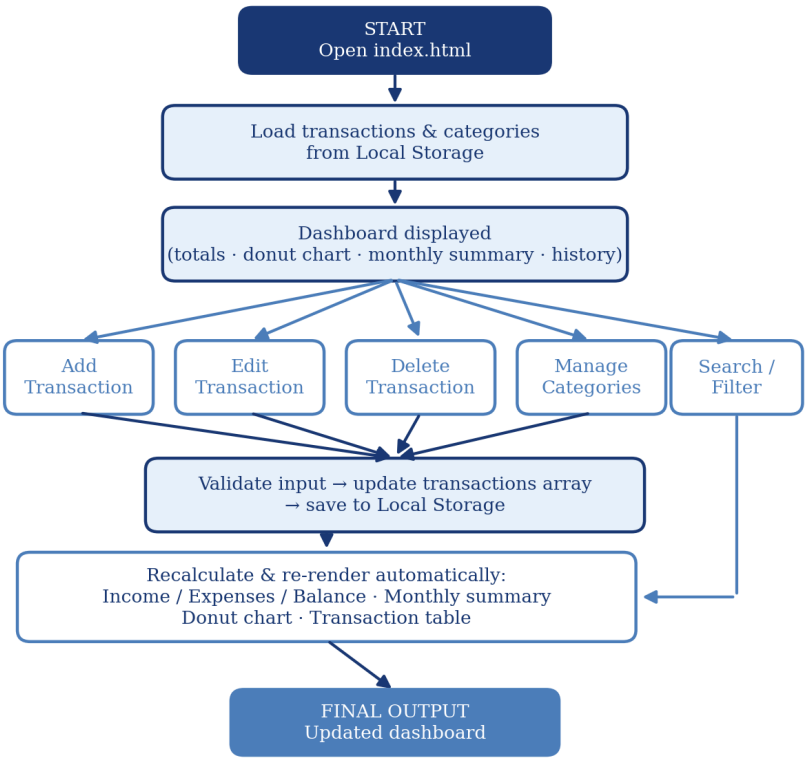


Figure 1: Application flow from the initial page to the final output

3.5 Data Structure

Each transaction is saved as one object in a list. The list is stored in Local Storage as JSON text. The fields are shown below.

Field	Meaning
id	A unique number made from the current time. It is used to find a record for edit or delete.
type	Shows if the record is income or expense.
amount	The money value. It must be more than zero.
date	The date of the record in YYYY-MM-DD format.
category	The group of the record, such as Food or Rent.
description	A short note about the record.

3.6 System Requirements

- Any modern web browser such as Chrome, Edge or Firefox.
- A computer or mobile phone. No internet or server is needed.
- A text editor such as VS Code to open or change the code.

4. Practical Implementation

The project was made in simple steps. First, the HTML page was made with the form, chart, tables and filters. Second, CSS was added for blue colours and layout. Third, JavaScript was added. Records are kept in a list and saved in Local Storage. One function, show(), redraws the totals, summary, chart and table every time data changes, so everything always matches.

4.1 Features Implemented

Feature	Purpose
Add / Edit / Delete	Keep records correct and up to date.
Income and expense	Save type, amount, date, category and description.
Categories	Add new categories or delete unused ones.
Search and filter	Find records by text, category, type and date.
Totals	Show income, expenses and balance automatically.
Monthly summary	Show income, expenses and net for each month.
Donut chart	Show expenses by category or month.
Local Storage	Keep data after refresh or closing the page.
Responsive design	Works on computer and mobile screens.

4.2 Source Code

The code is short and has simple comments inside it to explain each step.

HTML

HTML makes the page structure. It has the title, three cards, the form, the category box, the chart, the monthly table and the history table. Each important item has an id so JavaScript can use it.

```
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1">
<title>Personal Expense Tracker</title>
<link rel="stylesheet" href="style.css">
</head>
<body>
<!-- Page title -->
<header><h1>Personal Expense Tracker</h1></header>

<main>
  <!-- Three boxes: income, expense, balance -->
  <section class="cards">
    <div class="card"><span>Total Income</span><b id="inc">0</b></div>
    <div class="card"><span>Total Expenses</span><b id="exp">0</b></div>
    <div class="card"><span>Balance</span><b id="bal">0</b></div>
  </section>

  <div class="grid">
    <!-- Form to add or edit a transaction -->
    <section class="panel">
      <h2 id="formTitle">Add Transaction</h2>
      <form id="txForm">
        <input type="hidden" id="txId">
```

```
        <select id="type"><option value="expense">Expense</option><option
value="income">Income</option></select>
        <input type="number" id="amount" min="0.01" step="0.01" placeholder="Amount"
required>
        <input type="date" id="date" required>
        <select id="category"></select>
        <input type="text" id="desc" placeholder="Description" required>
        <button class="btn">Save</button>
      </form>

      <!-- Add or delete categories -->
      <h3>Categories</h3>
      <form id="catForm" class="inline">
        <input id="catName" placeholder="New category" required>
        <button class="btn">Add</button>
      </form>
      <ul id="catList" class="chips"></ul>
    </section>

    <!-- Donut chart -->
    <section class="panel">
      <h2>Expense Chart</h2>
      <select id="chartMode"><option value="category">By Category</option><option
value="month">By Month</option></select>
      <div class="donutWrap">
        <svg viewBox="0 0 42 42" id="donut"></svg>
        <div class="center"><small>Total Spent</small><b id="donutTotal">0</b></div>
      </div>
      <ul id="legend" class="legend"></ul>
    </section>
  </div>

  <!-- Monthly summary table -->
  <section class="panel">
    <h2>Monthly Summary</h2>

<table><thead><tr><th>Month</th><th>Income</th><th>Expenses</th><th>Net</th></tr></thead><
tbody id="monthBody"></tbody></table>
  </section>

  <!-- Search, filter and history table -->
  <section class="panel">
    <h2>Transaction History</h2>
    <div class="filters">
      <input id="fSearch" placeholder="Search description">
      <select id="fCategory"></select>
      <select id="fType"><option value="all">All Types</option><option
value="income">Income</option><option value="expense">Expense</option></select>
      <input type="date" id="fFrom" title="From date">
      <input type="date" id="fTo" title="To date">
      <button class="btn" id="clearBtn">Clear</button>
    </div>

<table><thead><tr><th>Date</th><th>Description</th><th>Category</th><th>Amount</th><th></t
h></tr></thead><tbody id="txBody"></tbody></table>
  </section>
</main>
<script src="script.js"></script>
</body>
</html>
```

CSS

CSS gives style to the page. It sets blue colours, styles the cards, forms and tables, puts the form and chart side by side, and changes to one column on small screens.

```
/* Colours */
:root { --dark:#1b3a73; --blue:#4f81bd; --light:#eaf1fa; --line:#c9d9ee; }

/* Page and fonts */
* { box-sizing: border-box; }
body { margin:0; background:var(--light); font-family:Georgia, serif; color:#222; }
header { background:var(--dark); color:#fff; text-align:center; padding:20px;
border-bottom:4px solid var(--blue); }
h1 { margin:0; }
main { max-width:1050px; margin:auto; padding:16px; }
```

```
/* Boxes (cards and panels) */
.cards { display:grid; grid-template-columns:repeat(3,1fr); gap:12px; margin-bottom:16px;
}
.card, .panel { background:#fff; border:1px solid var(--line); border-radius:4px;
padding:14px; }
.card { text-align:center; border-top:4px solid var(--blue); }
.card span { display:block; color:#5a6b85; }
.card b { font-size:1.5rem; color:var(--dark); }
.panel { margin-bottom:16px; }
.panel h2 { margin-top:0; color:var(--dark); border-bottom:2px solid var(--blue); }
h3 { color:var(--dark); }

/* Two columns: form  */
.grid { display:grid; grid-template-columns:1fr 1fr; gap:16px; }

/* Form fields and buttons */
input, select { width:100%; padding:8px; margin:0 0 8px; border:1px solid #aac3e3;
border-radius:3px; font:inherit; }
.btn { padding:8px 16px; background:var(--dark); color:#fff; border:0; border-radius:3px;
cursor:pointer; font:inherit; }
.btn:hover { background:var(--blue); }
.inline { display:flex; gap:8px; }

/* Category chips */
.chips { list-style:none; padding:0; display:flex; flex-wrap:wrap; gap:6px; }
.chips li { background:var(--light); border:1px solid var(--line); border-radius:14px;
padding:2px 4px 2px 10px; }
.chips button { border:0; background:none; color:var(--dark); cursor:pointer; }

/* Donut chart */
.donutWrap { position:relative; max-width:220px; margin:auto; }
#donut { width:100%; transform:rotate(-90deg); }
.center { position:absolute; inset:0; display:flex; flex-direction:column;
align-items:center; justify-content:center; }
.legend { list-style:none; padding:0; }
.legend li { display:flex; gap:8px; align-items:center; padding:2px 0; }
.legend i { width:12px; height:12px; display:inline-block; }
.legend span { flex:1; }

/* Filters and tables */
.filters { display:grid; grid-template-columns:2fr 1.3fr 1fr 1fr 1fr auto; gap:8px; }
table { width:100%; border-collapse:collapse; }
th { background:var(--dark); color:#fff; text-align:left; padding:8px; font-weight:normal;
}
td { padding:8px; border-bottom:1px solid var(--line); }
td button { border:0; background:none; color:var(--dark); text-decoration:underline;
cursor:pointer; }

/* Small screens: one column */
@media (max-width:800px) { .grid, .cards, .filters { grid-template-columns:1fr; } }
```

JavaScript

JavaScript makes the page work. It saves and loads data, adds, edits and deletes records, filters them, calculates totals, makes the monthly summary and draws the donut chart.

```
// ---------- 1. Data: load from Local Storage ----------
let transactions = JSON.parse(localStorage.getItem("tx")) || [];
let categories = JSON.parse(localStorage.getItem("cats")) || ["Food", "Transport", "Rent",
"Shopping", "Bills", "Health", "Salary", "Other"];
const COLORS = ["#0b2a5b", "#1f4e9c", "#4f81bd", "#7aa5d6", "#a9c6e8", "#2d6cb5",
"#163e7a", "#5c93cf"];

// Short helper names
const $ = id => document.getElementById(id);
const money = n => "₹" + n.toLocaleString("en-IN");

// Save everything to Local Storage
function save() {
  localStorage.setItem("tx", JSON.stringify(transactions));
  localStorage.setItem("cats", JSON.stringify(categories));
}

// ---------- 2. Add or edit a transaction ----------
$("txForm").onsubmit = e => {
  e.preventDefault();                                  // stop page reload
  const t = {
```

```
    id: $("txId").value || Date.now().toString(),      // old id = edit, no id = new
    type: $("type").value, amount: +$("amount").value,
    date: $("date").value, category: $("category").value, description: $("desc").value
  };
  const i = transactions.findIndex(x => x.id === t.id);
  if (i >= 0) transactions[i] = t; else transactions.push(t);
  e.target.reset(); $("txId").value = ""; $("formTitle").textContent = "Add Transaction";
  $("date").value = new Date().toISOString().slice(0, 10);   // set today again
  show();
};

// ---------- 3. Edit and delete buttons in the table ----------
$("txBody").onclick = e => {
  const id = e.target.dataset.id;
  if (!id) return;
  if (e.target.dataset.act === "del") {
    transactions = transactions.filter(t => t.id !== id);   // remove the record
  } else {
    const t = transactions.find(x => x.id === id);          // put the record back in the
form
    $("txId").value = id; $("type").value = t.type; $("amount").value = t.amount;
    $("date").value = t.date; $("category").value = t.category; $("desc").value =
t.description;
    $("formTitle").textContent = "Edit Transaction";
  }
  show();
};

// ---------- 4. Categories ----------
$("catForm").onsubmit = e => {
  e.preventDefault();
  const name = $("catName").value.trim();
  if (!categories.includes(name)) categories.push(name);    // no duplicates
  e.target.reset(); show();
};
$("catList").onclick = e => {
  const c = e.target.dataset.cat;
  if (!c) return;
  if (transactions.some(t => t.category === c)) alert("This category is in use.");
  else categories = categories.filter(x => x !== c);
  show();
};

// ---- 5. Search and filter----
function filtered() {
  const q = $("fSearch").value.toLowerCase();
  return transactions.filter(t =>
    t.description.toLowerCase().includes(q) &&
    ($("fCategory").value === "all" || t.category === $("fCategory").value) &&
    ($("fType").value === "all" || t.type === $("fType").value) &&
    (!$("fFrom").value || t.date >= $("fFrom").value) &&
    (!$("fTo").value || t.date <= $("fTo").value)
  ).sort((a, b) => b.date.localeCompare(a.date));          // newest first
}
["fSearch", "fCategory", "fType", "fFrom", "fTo"].forEach(id => $(id).oninput = show);
$("clearBtn").onclick = () => {
  ["fSearch", "fFrom", "fTo"].forEach(id => $(id).value = "");
  $("fCategory").value = $("fType").value = "all"; show();
};
$("chartMode").onchange = show;

// ---------- 6. Donut chart ----------
function drawDonut() {
  const byMonth = $("chartMode").value === "month";
  const sums = {};
  transactions.filter(t => t.type === "expense").forEach(t => {
    const key = byMonth ? t.date.slice(0, 7) : t.category;  // group by month or category
    sums[key] = (sums[key] || 0) + t.amount;
  });
  const total = Object.values(sums).reduce((a, b) => a + b, 0);
  $("donutTotal").textContent = money(total);

  let start = 0, svg = "", legend = "";
  Object.entries(sums).forEach(([name, value], i) => {
    const pct = value / total * 100;
    // One circle per slice. Circle length is 100, so dash = percentage.
    svg += `<circle cx="21" cy="21" r="15.9155" fill="none" stroke="${COLORS[i % 8]}"
stroke-width="6"
            stroke-dasharray="${pct} ${100 - pct}" stroke-dashoffset="${-start}"/>`;
```

```
    legend += `<li><i style="background:${COLORS[i %
8]}"></i><span>${name}</span><b>${money(value)}</b> (${pct.toFixed(1)}%)</li>`;
    start += pct;
  });
  $("donut").innerHTML = svg;
  $("legend").innerHTML = legend;
}

// ---------- 7. category list ----------
function show() {
  save();

  // Category lists (form, filter and chips)
  const opts = categories.map(c => `<option>${c}</option>`).join("");
  const old = $("fCategory").value;
  $("category").innerHTML = opts;
  $("fCategory").innerHTML = `<option value="all">All Categories</option>` + opts;
  $("fCategory").value = categories.includes(old) ? old : "all";
  $("catList").innerHTML = categories.map(c => `<li>${c}<button
data-cat="${c}">×</button></li>`).join("");

  // Total income, total expenses and balance
  const sum = type => transactions.filter(t => t.type === type).reduce((s, t) => s +
t.amount, 0);
  $("inc").textContent = money(sum("income"));
  $("exp").textContent = money(sum("expense"));
  $("bal").textContent = money(sum("income") - sum("expense"));

  // Monthly summary
  const m = {};
  transactions.forEach(t => {
    const k = t.date.slice(0, 7);
    m[k] = m[k] || { income: 0, expense: 0 };
    m[k][t.type] += t.amount;
  });
  $("monthBody").innerHTML = Object.keys(m).sort().reverse().map(k =>

`<tr><td>${k}</td><td>${money(m[k].income)}</td><td>${money(m[k].expense)}</td><td>${money
(m[k].income - m[k].expense)}</td></tr>`).join("");

  // History table
  $("txBody").innerHTML = filtered().map(t =>
    `<tr><td>${t.date}</td><td>${t.description}</td><td>${t.category}</td>
     <td>${t.type === "income" ? "+" : "-"}${money(t.amount)}</td>
     <td><button data-act="edit" data-id="${t.id}">Edit</button><button data-act="del"
data-id="${t.id}">Delete</button></td></tr>`).join("");

  drawDonut();
}

// ---------- 8. Start ----------
$("date").value = new Date().toISOString().slice(0, 10);   // today's date
show();
```

Main Functions Explained

Function / part	What it does
save()	Writes the records and categories to Local Storage.
txForm.onsubmit	Reads the form, adds a new record or updates an old one, then calls show().
txBody.onclick	Handles the Edit and Delete buttons in the history table.
catForm / catList	Adds a new category or deletes an unused category.
filtered()	Returns only the records that match the search, category, type and dates.
drawDonut()	Adds the expenses by category or month and draws one circle slice for each.

Function / part	What it does
show()	Updates the categories, totals, monthly summary, history table and chart.

4.3 Project Execution

To run the application:

1. Keep index.html, style.css and script.js in one folder.
2. Open index.html in a browser such as Chrome or Edge. No installation is needed.
3. Choose the type, enter amount, date, category and description, then click Save.
4. See the totals, monthly summary and chart update at once.
5. Use Edit, Delete, search or filters to manage records.
6. Refresh the page to check that the data is still saved.

How it works: user enters data → data is saved in the list and Local Storage → show() updates totals, summary, chart and table → the user sees the new result.

5. Output Screenshots

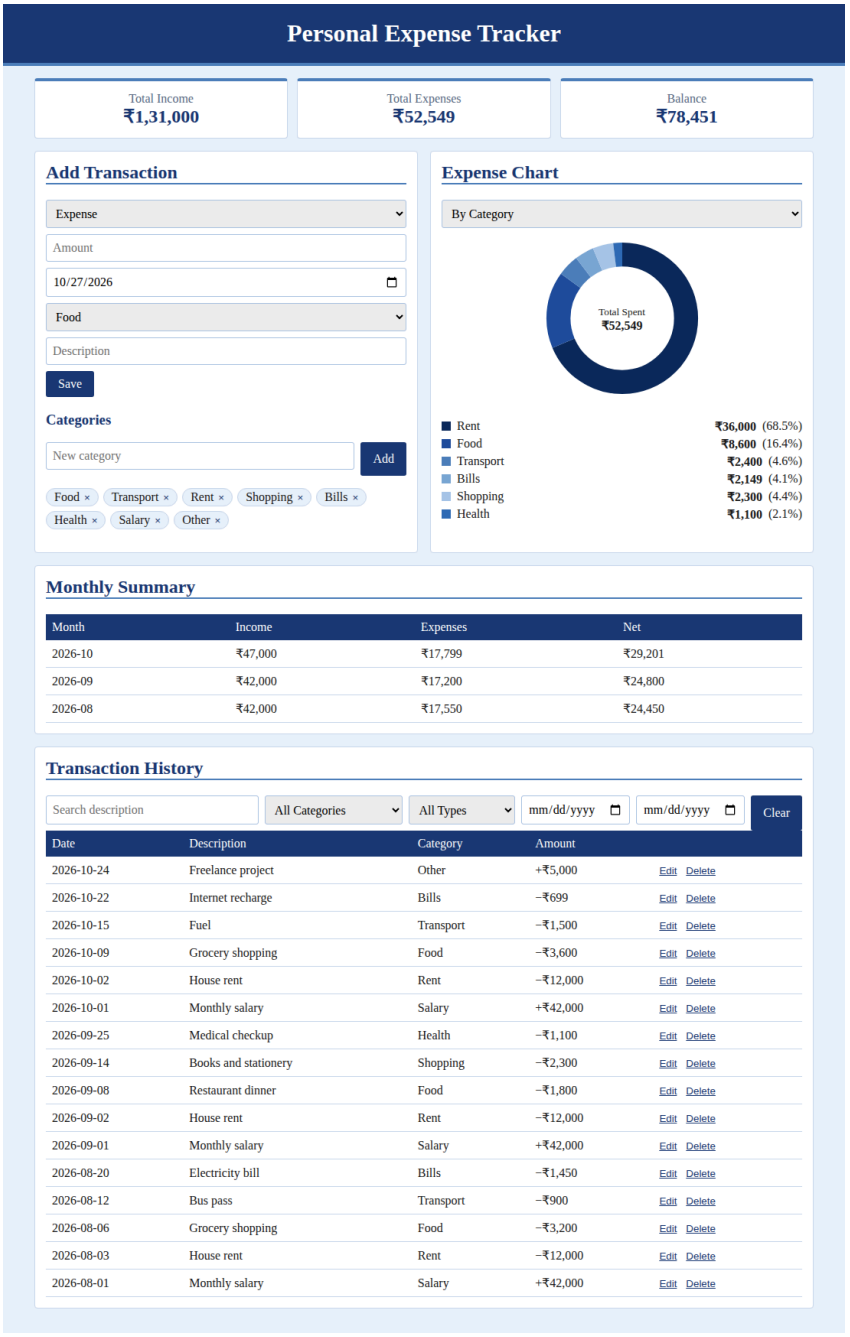


Figure 2: Dashboard with totals, donut chart, monthly summary and history

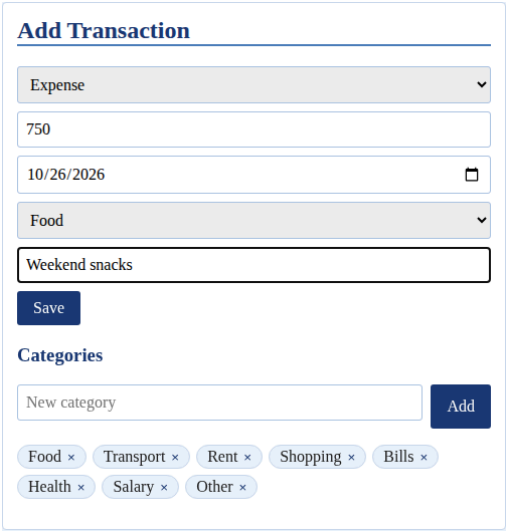


Figure 3: Adding a new transaction

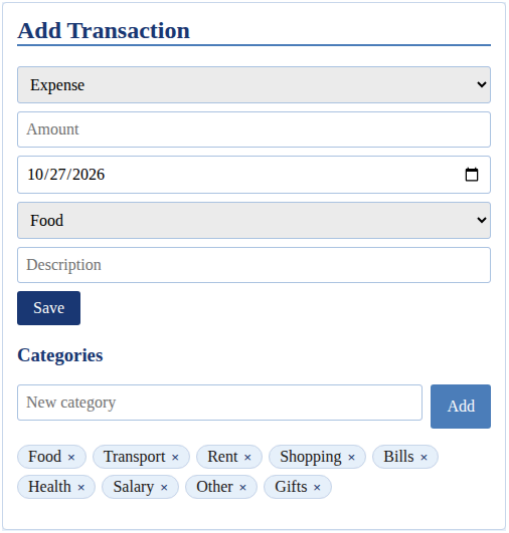


Figure 4: Adding a new category

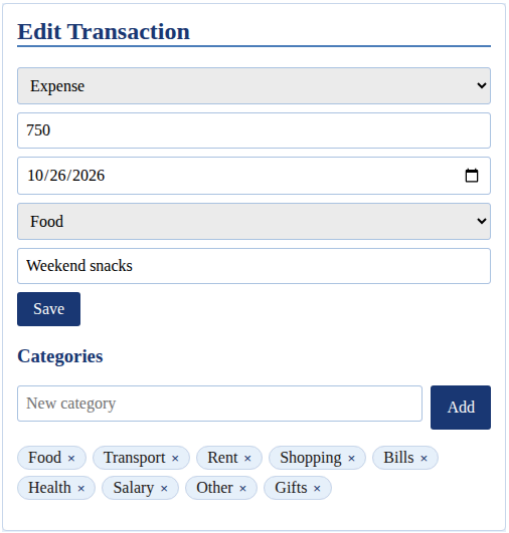


Figure 5: Editing an existing transaction



Figure 6: Search and filter results

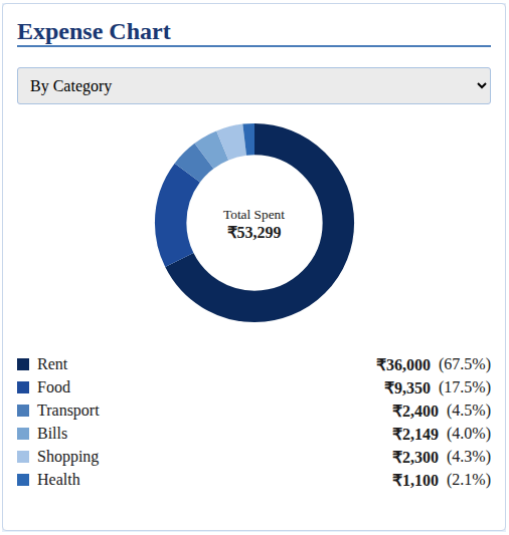


Figure 7: Donut chart of expenses by category

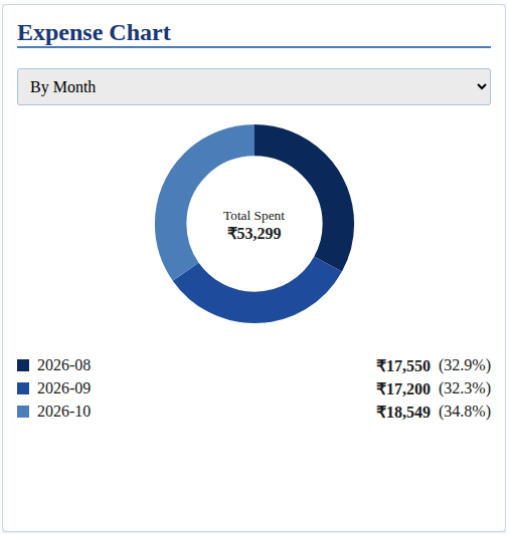


Figure 8: Donut chart of expenses by month

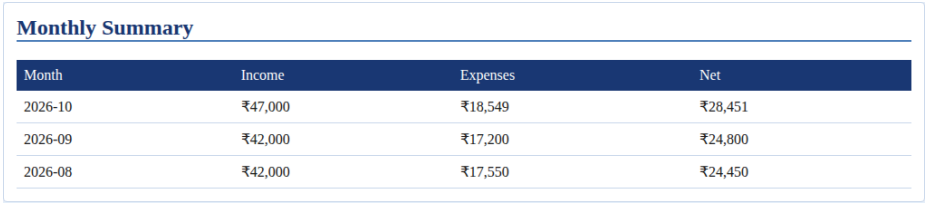


Figure 9: Monthly financial summary

6. Technical Explanation

- **Data:** Each record is an object with id, type, amount, date, category and description. Records are saved as JSON in Local Storage.
- **Events:** Submit and click events change the data and then call show() to update the screen.
- **Array methods:** filter(), reduce(), find(), sort() and map() are used for searching, totals, grouping and building the HTML.
- **Monthly grouping:** Month totals are grouped using the first 7 letters of the date (YYYY-MM).
- **Donut chart:** An SVG circle with radius 15.9155 has a length of 100. So each slice uses its percentage as the dash length, and the slices together make a donut.
- **Checks:** The amount and date are required, duplicate categories are not added, and categories in use cannot be deleted.

6.1 Testing

The application was tested with the following cases and all gave the correct result.

Test	Expected result
Add an expense of 750 in Food	A new row appears; expenses and balance change; the chart updates.
Edit a record and change the amount	The row shows the new amount; totals change.
Delete a record	The row is removed; totals, summary and chart update.
Search the word "grocery"	Only records with this word are shown.
Filter by type and by date range	Only matching records are shown.
Add a category and try to delete a used one	New category is added; a used category is not deleted.
Refresh the browser	All records are still there (Local Storage).
Switch the chart to By Month	The donut shows one slice for each month.

6.2 Advantages

- Simple to use and fast.
- Works without internet and without installing anything.
- Data stays private in the user's own browser.
- Easy to understand and extend because the code is short.

6.3 Limitations

- Data is saved only in one browser on one device.
- There is no login, so anyone using the same browser can see the data.
- Clearing browser data will delete the records.

6.4 Programming Concepts Used

- **Basics:** Variables, constants and arrow functions are used to keep the code short.
- **DOM:** The page is read and changed using document.getElementById and innerHTML.
- **Events:** onsubmit, onclick and oninput run code when the user acts.
- **Arrays and objects:** Records are stored as an array of objects and kept as JSON.
- **Template strings:** Template strings (backticks) build table r7ows and chart slices.
- **Browser storage:** localStorage.setItem and getItem keep data after the page is closed.

Together these concepts show how a complete small application can be made without any library or framework.

7. Conclusion

The Personal Expense Tracker has all the required features: add, edit and delete records, categories, search and filter, totals, monthly summary, Local Storage and a donut chart that updates when data changes. In this project I learned DOM handling, events, JSON, Local Storage, array methods, SVG charts and responsive CSS. It is a simple and useful tool to understand personal spending.

8. Future Enhancements

- Budget limit for each category with a warning.
- Export data to CSV or PDF.
- Login and cloud backup to use on many devices.
- Repeat payments and reminders.
- More charts such as yearly and income-vs-expense charts.
- Dark mode, more currencies and better data security.

- Better security: A PIN or password protects the data, and the data is saved in encrypted form.
- More charts: Yearly charts and income-versus-expense charts help the user see spending trends. To what's the money goes off.